ЗЕРТХАНАЛЫҚ ЖҰМЫСТАР ЕСЕБІ

StayBook

Қонақүй нөмірлерін брондау ақпараттық жүйесі

№1 · №2 · №3 · №4 · №5 · №6 · №7

Білім алушы: Касанова Аружан

Тобы: Ис-24а

Оқытушы: _____

Тапсыру күні: _____

Python · FastAPI · SQLModel · SQLite
Streamlit · Docker Compose · pytest

Есептің құрамы

- Жоба туралы
- №1 зертханалық жұмыс
- №2 зертханалық жұмыс
- №3 зертханалық жұмыс
- №4 зертханалық жұмыс
- №5 зертханалық жұмыс
- №6 зертханалық жұмыс
- №7 зертханалық жұмыс
- Қорғауға дайындық

Бұл есеп бір ақпараттық жүйені жобалауды, дерекқор мен толық CRUD API жасауды, Streamlit интерфейсін, Docker арқылы іске қосуды және автоматты тестілеуді қамтитын жеті зертханалық жұмысты біріктіреді.

Титул бетіндегі оқытушы мен тапсыру күні жолдары тапсырар алдында толтырылады.

№1–7 зертханалық жұмыстар бойынша есеп

Пән: Прикладное программное обеспечение информационных систем (2026 г.)

Жоба: StayBook — қонақүй нөмірлерін брондау жүйесі

Орындаған студент: Касанова Аружан

Топ: Ис-24а

Тексерген оқытушы: _____

Тапсыру күні: _____

Репозиторий: https://github.com/kasanovaaruzan74-oss/StayBook

Жобаның мақсаты және пәндік сала

StayBook қонақүйдегі нөмірлер, олардың санаттары, қонақтар мен брондаулар туралы мәліметті бір жүйеде сақтайды. Жүйені қонақүй әкімшісі пайдаланады. Деректерді қосу, оқу, өзгерту, жою және сүзгілеу операциялары API мен веб-интерфейс арқылы орындалады. Нөмірді брондау кезінде күндердің дұрыстығы, байланысты жазбалардың бар болуы және мерзімдердің қабаттаспайтыны тексеріледі.

Жеті зертхана бір жобаның кезеңдері ретінде орындалады: архитектура → база → API → толық бизнес логика → веб-интерфейс → контейнерлер → автоматты тесттер және қорғау.

Қолданылған құралдар

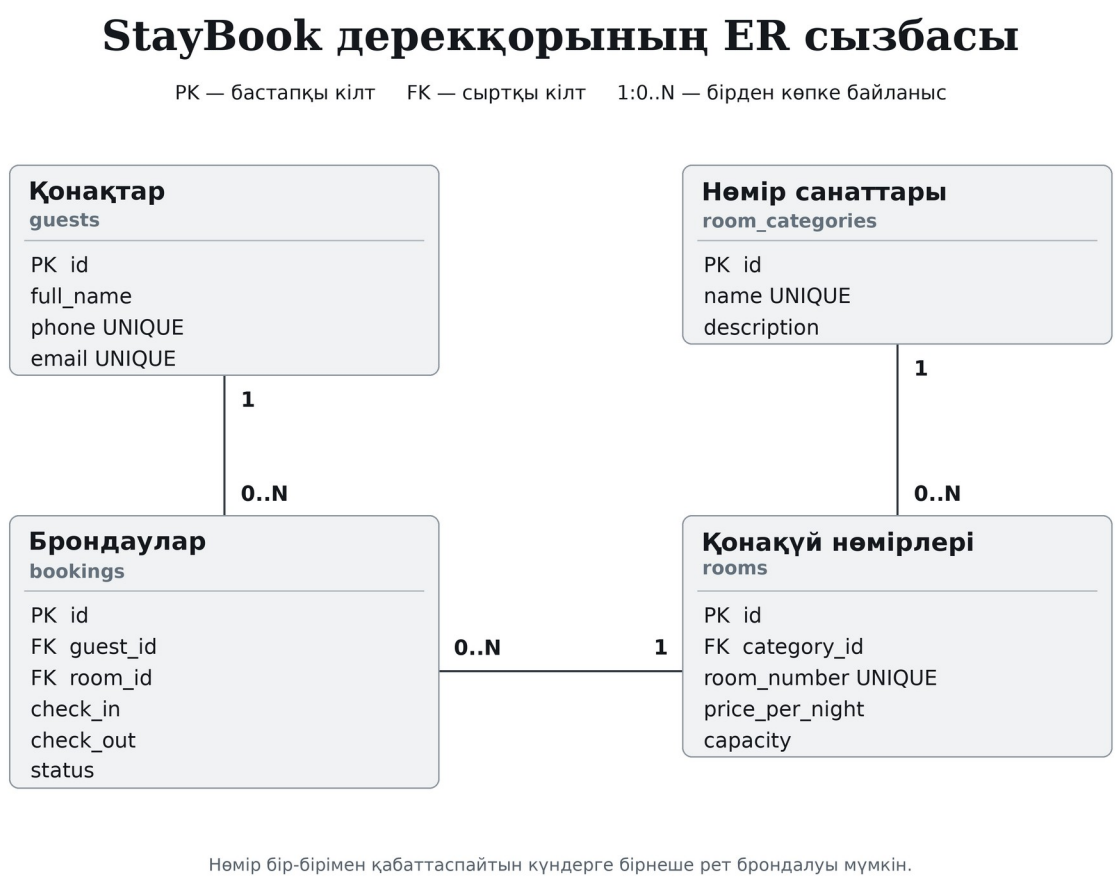
Құрал	Міндеті
Python 3.11+	Негізгі бағдарламалау тілі
FastAPI, Uvicorn	HTTP маршруттары және ASGI сервері
SQLModel, SQLAlchemy	ORM кестелері, байланыстар, транзакциялар
SQLite	Жергілікті реляциялық дерекқор
Pydantic	Сұрау және жауап схемалары, валидация
Streamlit, Requests	Веб-интерфейс және API-мен HTTP/JSON байланысы
Docker, Docker Compose	Екі компонентті контейнерлеу, ортақ желі және volume
Pytest, TestClient, HTTPX	API мен бизнес шарттарын автоматты тексеру
Git, GitHub	Нұсқалар тарихы және қашықтағы репозиторий
Codex ИИ көмекшісі	Құрылым, код, тест сценарийлерін дайындау және код ревьюі

№1. Ақпараттық жүйені жобалау және орта

Пәндік сала ретінде қонақүй нөмірлерін брондау таңдалды. Басты әрекеттер: қонақты тіркеу, санат пен нөмірді қосу, баға және сыйымдылықты сақтау, келу/кету мерзіміне брондау жасау.

Төрт кесте жобаланды:

1. `guests` — қонақтың аты-жөні, бірегей телефон нөмірі және электрондық поштасы.
2. `room_categories` — нөмір санаты мен оның сипаттамасы.
3. `rooms` — санатқа тиесілі нөмір, түндік баға және сыйымдылық.
4. `bookings` — қонақ пен нөмірге сілтемелер, келу/кету күндері және мәртебе.



StayBook дерекқорының ER сызбасы

1-сурет — ER-диаграмма. Бастапқы кілт `PK`, сыртқы кілт `FK`, бірегей өріс `UNIQUE` арқылы белгіленген. Бір санатқа бірнеше нөмір, бір қонаққа бірнеше брондау және бір нөмірге әртүрлі күндердегі бірнеше брондау сәйкес келеді.

Сызба PNG және SVG форматында, өңдеуге арналған draw.io нұсқасымен бірге берілді. Жоба `app/database/`, `app/routers/`, `app/frontend/`, `tests/`, `docs/` және `scripts/` бөлімдеріне бөлінді. Тәуелділіктер файлдары, README мен `.gitignore` дайындалды.

Жергілікті орта `python -m venv venv` арқылы құрылады. Cursor/VS Code орнату мен ИИ кеңейткішін баптау студенттің өз компьютерінде жасалады; README-де жүктеу, ортаны белсендіру және іске қосу қадамдары берілген.

№2. Реляциялық база және ORM

ER-сызба `app/database/models.py` файлындағы SQLModel кластарына көшірілді. Модельдер бастапқы және сыртқы кілттерді, бірегей мәндерді және `Relationship` байланыстарын сипаттайды. Баға `Decimal`, брондау күндері `date` типімен беріледі.

`app/database/db.py` engine-ді, кестелерді дайындайтын `init_db()` және сұрауға сессия беретін `get_session()` функцияларын қамтиды. SQLite сыртқы кілттерді тексереді. Жергілікті базаның әдепкі файлы — `database.db`. Бұрынғы импорттар үшін `app/db.py` және `app/models.py` үйлесімділік модульдері сақталған.

`seed.py` скрипті әр кестеге 5 мысал жазбадан енгізуге арналған. Бірдей деректерді қайталап қоспау үшін табиғи бірегей мәндер тексеріледі:

```
python seed.py
```

Бұл команда жаңа базаға бастапқы деректер енгізуге арналған. Кейінгі іске қосуларда деректері бар базаны қозғамау үшін `python seed.py --if-empty` қолданылады.

`database.db`, виртуалды орта, кэштер мен құпия конфигурациялар Git-ке қосылмайды. Қайта қалпына келтіруге қажет модельдер мен seed коды репозиторийде сақталады.

№3. FastAPI сервері және GET/POST

Қолданбаның кіру нүктесі — `app/main.py`. Сервер:

```
uvicorn app.main:app --reload
```

Жүйеде нөмірлер, қонақтар, санаттар және брондаулар үшін маршруттар бар. №3 жұмыстың негізгі әрекеттері — нөмірлер тізімін оқу, ID арқылы табу және жаңа нөмір қосу. Әр сұрау `Depends(get_session)` арқылы өз дерекқор сессиясын алады.

Сұрау схемалары ORM кестелерінен бөлінген. Нөмір қосқанда баға, бос белгі, санаттың бар болуы және сыйымдылық тексеріледі. Сәтті жасау `201`, жарамсыз дерек `422`, табылмаған ID `404`, қайталанатын бірегей мән `409` мәртебесімен беріледі.

Swagger UI мекенжайы — <http://localhost:8000/docs>. Құжаттама маршруттар мен схемалардан автоматты жасалады.



StayBook Swagger UI

2-сурет — API-дің Swagger UI көрінісі. Орындалған нақты сұраулар мен тексерулер бөлек тексеру есебінде тіркеледі.

№4. Толық CRUD және бизнес логика

`PATCH /rooms/{room_id}` тек жіберілген өрістерді өзгертеді. `model_dump(exclude_unset=True)` жіберілмеген өрістерді сақтайды, ал міндетті өріске нақты `null` жіберу валидациямен қабылданбайды.

`DELETE /rooms/{room_id}` нөмірдің бар болуын және байланысты брондауларын тексереді. Сәтті жою `204` бос жауабын береді. Жазба жоқ болса `404`, байланысты деректі бұзатын әрекет болса `409` қайтарылады.

Тізімді сүзгілеу query параметрлерімен беріледі:

```
GET /rooms?search=10&min_price=10000&max_price=30000&category_id=1
```

SQL сұраулары SQLModel конструкцияларымен жасалады; пайдаланушы енгізуі SQL мәтініне тікелей жалғанбайды. Төменгі және жоғарғы баға шектері тексеріледі.

Брондауда `check_out` күні `check_in` күнінен кейін болуы керек. Қонақ пен нөмір базада болуы тиіс. Бір нөмірге қабаттасатын, күшін жоймаған брондау қабылданбайды. `[check_in, check_out)` интервалы бойынша бір қонақтың шығу күні келесісінің келу күнімен сәйкес келуі мүмкін.

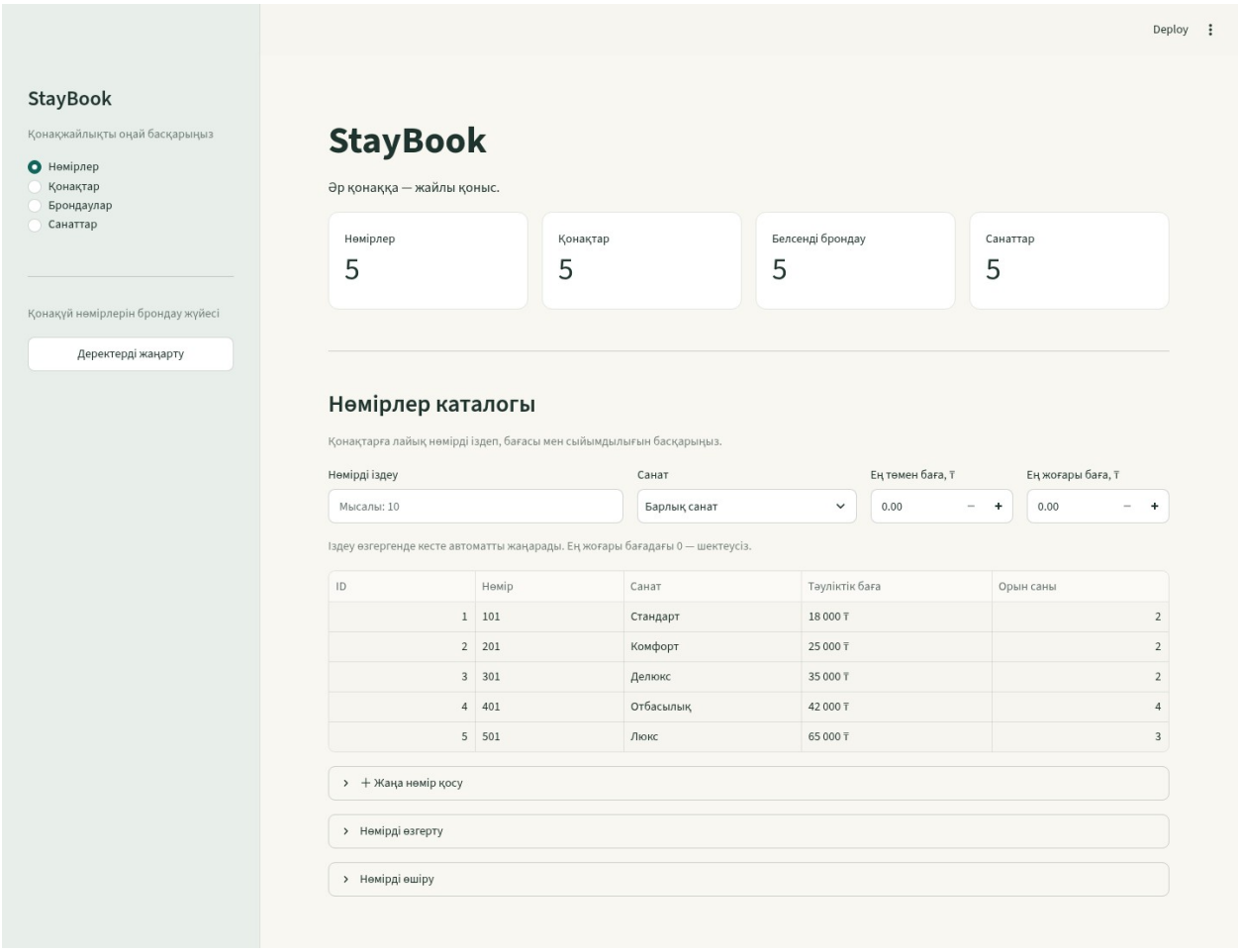
№5. Streamlit веб-интерфейсі

Негізгі интерфейс — `app/frontend/ui.py`. API жұмыс істеп тұрған кезде екінші терминалда:

```
streamlit run app/frontend/ui.py
```

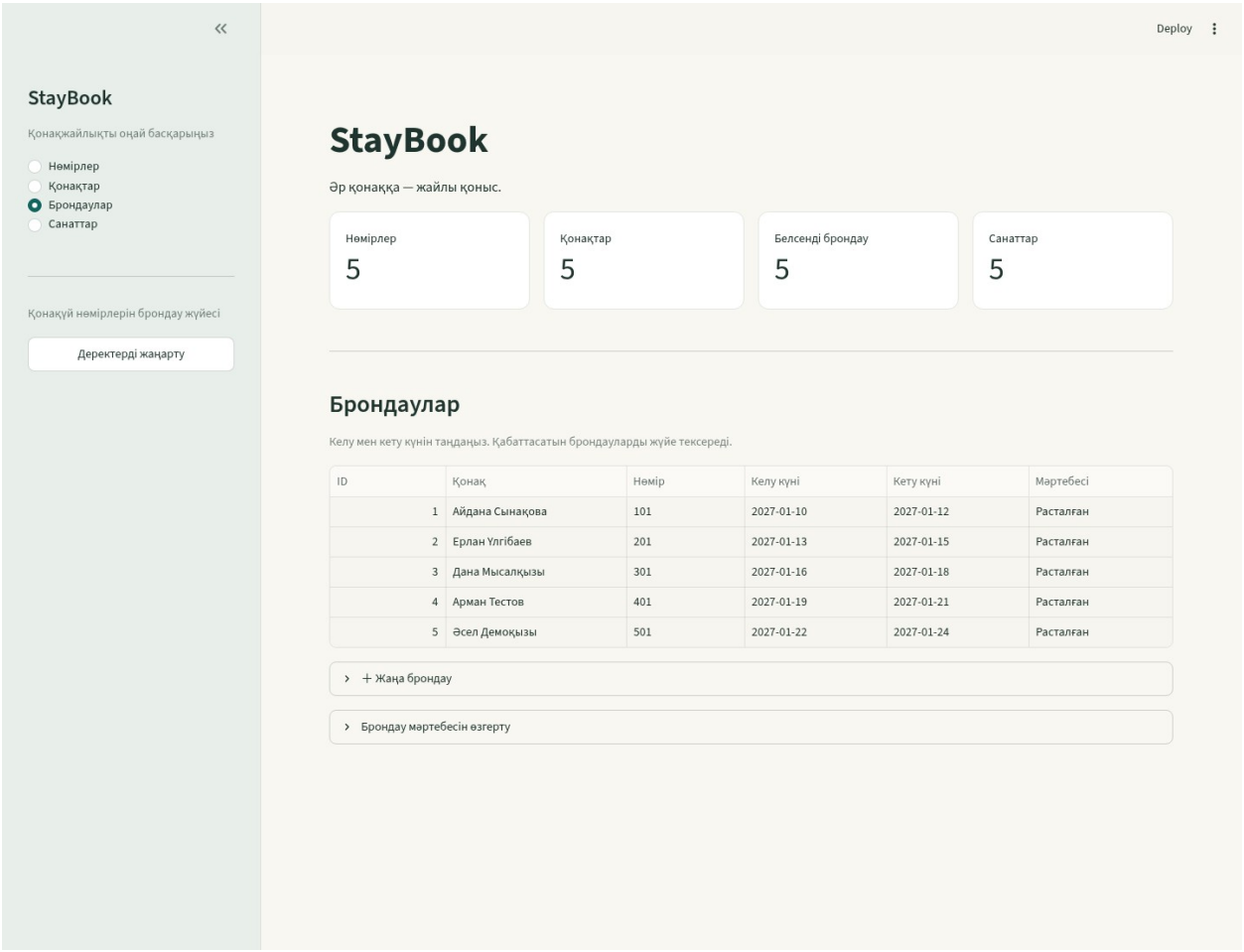
Браузерде <http://localhost:8501> ашылады. UI қонақүй нөмірлерін, брондауларды, қонақтарды және санаттарды көрсетеді. `Requests` кітапханасы API-ге HTTP/JSON арқылы қатынасады. Іздеу мен баға шарттары GET сұрауының `params` параметрімен, жаңа жазба өрістері POST сұрауының `json` денесімен жіберіледі.

Іздеу виджетінің мәні өзгеріп, Streamlit-ке жіберілгенде скрипт қайта орындалып, тізім жаңа шартпен жүктеледі. Қосу формалары, өзгерту және жою әрекеттері сервердің CRUD маршруттарын пайдаланады. Сәтті әрекет туралы хабарлама мен сервер қатесінің түсіндірмесі интерфейсте көрсетіледі.



StayBook Streamlit интерфейсі

3-сурет — нөмірлер тізімі және пайдаланушы әрекеттері. UI дерекқорды тікелей ашпайды; дерек өзгерісі FastAPI арқылы өтеді.



Бронданулар бөлімі

4-сурет — брондаулар бөлімі. `BACKEND_URL` орта айнымалысы API адресін анықтайды, әдепкісі — `http://127.0.0.1:8000`. Бұрынғы `frontend/app.py` іске қосу файлы үйлесімділік үшін сақталған.

№6. Docker және Docker Compose

`Dockerfile` FastAPI-ді, `Dockerfile.frontend` Streamlit-ті бөлек образдарға жинайды. Тәуелділіктер кодтан бұрын орнатылады, бұл құрастыру кэшін пайдалануға мүмкіндік береді. `.dockerignore` жергілікті орта, база, кэш және қажетсіз файлдарды образға қоспайды.

`docker-compose.yml` екі сервисті байланыстырады:

Сервис	Мақсаты	Байланыс
`backend`	API, SQLite, бастапқы деректер	Хост `8000` → контейнер `8000`
`frontend`	Веб-интерфейс	Хост `8501` → контейнер `8501`

Frontend контейнерінде `BACKEND_URL=http://backend:8000` беріледі. Compose желісінің ішкі DNS жүйесі `backend` атын тиісті контейнер адресіне сәйкестендіреді. `depends_on` және healthcheck backend дайын болғаннан кейін UI-ді бастауға мүмкіндік береді.

SQLite `/data/database.db` ішінде, тұрақты named volume-да сақталады. Контейнер іске қосылғанда `python seed.py --if-empty` төрт кесте де бос болса ғана мысал деректерді енгізеді. Қайта іске қосу бұрынғы бизнес деректерін өзгертпейді және жойылған мысалдарды өздігінен қалпына келтірмейді.

Іске қосу:

```
git clone https://github.com/kasanovaaruzan74-oss/StayBook.git
cd StayBook
docker compose up --build
```

Алдын ала Docker Desktop іске қосылып, `8000` және `8501` порттары бос болуы керек. Қалыпты `docker compose down` volume-ды сақтайды. Деректерді сақтау қажет кезде `down -v` қолданылмайды.

№7. Автоматты тесттер және қорғау

`tests/` ішінде Pytest және FastAPI TestClient қолданатын API сценарийлері берілген. Негізгі талаптар — GET `200` және жарамды POST `201`. Қосымша сценарийлер CRUD, валидация, іздеу, бірегей мәндер, сыртқы кілттер, жою шектеулері және брондау қайшылықтарын қамтиды.

Тесттер бөлек тестілік дерекқорды қолданады. TestClient сұраулары үшін Uvicorn-ды жеке іске қосу қажет емес:

```
python -m pip install -r requirements-dev.txt
pytest
```

Кодты тексеруде сессиялардың жабылуы, қате транзакцияны rollback жасау, параметрленген SQL шарттары, PATCH null валидациясы және брондаулардың қатар жазылуы қарастырылады. ИИ ұсынған код нақты логика және тест нәтижесімен тексеріледі.

README нұсқаулықтағы бес міндетті бөлімге сәйкес толтырылды. Әр зертханаға жеке түсіндірме және барлығы 35 бақылау сұрағының жауабы берілген. Автор мен топ көрсетілді; оқытушы мен тапсыру күніне арналған орындар сақталды.

Тексеру жағдайлары және нақты нәтижелер

Кесте күтілетін мінез-құлықты сипаттайды. Нақты тест саны, орындалған браузер сұраулары және Docker тексеруінің қорытындысы verification.md файлында беріледі.

Әрекет	Күтілетін нәтиже
Нөмірлер тізімін алу	`200`, тізім
Жарамды нөмір қосу	`201`, жасалған жазба
Теріс бағамен нөмір қосу	`422`
Бағаны PATCH арқылы өзгерту	`200`, басқа өрістер сақталады
Нөмір белгісі және баға бойынша сүзгілеу	Шартқа сәйкес жазбалар
Брондаусыз нөмірді жою	`204`, бос жауап
Жойылған ID-ге қайта жүгіну	`404`
Бірегей нөмір белгісін қайталау	`409`
Мерзімі қабаттасатын брондау	`409`
База толы кезде автоматты seed	Бұрынғы деректер өзгермейді
UI іздеу және форма жіберу	API арқылы жаңарған кесте
Compose арқылы екі сервис	UI және Swagger қолжетімді
Қалыпты down/up	Volume-дағы деректер сақталады

Git тарихы

Зертхана кезеңдерінің коммит тақырыптары:

```
feat: initial project structure with AI assist
feat: add SQLModel ORM models and database seed script
feat: implement FastAPI endpoints for GET and POST with validation
feat: complete CRUD operations with PATCH, DELETE and query filtering
feat: design interactive web UI using Streamlit integrated with FastAPI
chore: containerize application with Docker and Docker Compose
test: add integration tests and final documentation update
```

Нақты тарих `git log --oneline` арқылы қаралады. Коммиттер өздерінің нақты уақытын сақтайды. GitHub код пен құжаттарды сақтайды; веб-сервистер Docker не жергілікті Python командаларымен іске қосылады.

Қорғау реті

1. ER-сызбаны және кестелер арасындағы үш байланысты түсіндіру.
2. `docker compose up --build` арқылы жүйені қосып, сервис күйін көрсету.
3. UI-де іздеу, нөмір қосу, бағасын өзгерту және жоюды көрсету.
4. Қайшы брондау, теріс баға және жоқ ID жауаптарын түсіндіру.
5. Swagger-ді ашып, HTTP әдістері мен жауап схемаларын көрсету.
6. `pytest` орындап, нақты нәтижені қарау.
7. Негізгі файлдардың міндетін және Git тарихындағы кезеңдерді түсіндіру.

Бақылау сұрақтарының жауаптары: defense_questions.md. Әр жұмыстың толық орындау жолы: `lab_01.md`–`lab_07.md`.

Зертханалық жұмыс №1

Тақырып: ақпараттық жүйені жобалау, ИИ көмегімен әзірлеу ортасын және нұсқаларды басқаруды дайындау.

Жоба: StayBook — қонақүй нөмірлерін брондау жүйесі.

Мақсат

Пәндік саланы кестелерге бөлу, ER-диаграмма құру, Python жобасының құрылымын дайындау және Git арқылы өзгерістерді сақтау тәртібін түсіну.

1. Пәндік саланы анықтау

Қонақүй қызметкері қонақтарды, нөмір санаттарын, нөмірлердің бағасы мен сыйымдылығын және брондауларды есепке алады. Нөмір бірнеше рет брондалуы мүмкін, бірақ күшін жоймаған брондаулардың күндері қабаттаспайды.

Төрт негізгі мән таңдалды:

- `guests` — қонақтар;
- `room_categories` — нөмір санаттары;
- `rooms` — қонақүй нөмірлері;
- `bookings` — брондаулар.

Байланыстар: `room_categories → rooms`, `guests → bookings`, `rooms → bookings`. Барлығы — «бірден көпке»: `1 — 0..N`.

2. ER-диаграмма

Сызбадағы кестелердің қазақша тақырыбы, ағылшынша техникалық атауы және өрістері көрсетілген. `PK id` әр жазбаны анықтайды. `FK category_id`, `FK guest_id`, `FK room_id` кестелерді байланыстырады.

Тапсырылатын файлдар: `er_diagram.png`, масштабталатын `er_diagram.svg` және draw.io ішінде өңделетін `er_diagram.drawio`.

3. Git репозиторийі

Қашықтағы ашық репозиторий: kasanovaaruzan74-oss/StayBook (https://github.com/kasanovaaruzan74-oss/StayBook). Негізгі сипаттама `README.md` файлында беріледі, Python ортасы мен уақытша файлдар `.gitignore` арқылы алынып тасталады.

Жергілікті жүктеу:

```
git clone https://github.com/kasanovaaruzan74-oss/StayBook.git
cd StayBook
```

Қолданыстағы `StayBook` репозиторийі пайдаланылады. Нұсқаулықтағы `applied-is-project-2026` — атау стилінің мысалы; бұл жобаның нақты атауы — `StayBook`.

4. Жергілікті IDE және орта

Студент өз компьютеріне Cursor немесе VS Code және ИИ көмекшісін орнатады, жоба папкасын ашады. IDE орнату студенттің компьютерінде орындалатын қадам; репозиторийдегі код бұл орнатуды алмастырмайды.

```
python -m venv venv
```

Windows PowerShell: `.\venv\Scripts\Activate.ps1`. Linux/macOS: `source venv/bin/activate`.

```
python -m pip install -r requirements-dev.txt
```

ИИ-ге беруге болатын сұрау үлгісі:

> *Python тілінде қонақүй нөмірлерін брондау жүйесін жасаймын. FastAPI сервері, SQLModel дерекқоры және Streamlit интерфейсі үшін түсінікті папка құрылымын ұсын. fastapi, uvicorn, sqlmodel, streamlit тәуелділіктерін көрсет. Әр файлдың міндетін түсіндір.*

Жобада `app/database/`, `app/routers/`, `app/frontend/`, `tests/`, `docs/`, `scripts/` бөлімдері қарастырылған. Толық қызметтер кейінгі зертханаларда осы құрылымға қосылады.

5. Өзгерістерді сақтау

```
git add README.md .gitignore requirements.txt requirements-dev.txt app frontend docs scripts
git commit -m "feat: initial project structure with AI assist"
git push origin main
```

Командалар орындау тәртібін көрсетеді. Алдын ала жасалған коммиттерді қайталау қажет емес; нақты тарих `git log --oneline` арқылы қаралады.

Қорғауда көрсету

ER-диаграммадағы үш байланысты түсіндіру, виртуалды ортаны белсендіру, `.gitignore` және README файлын ашу. Қорытынды бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №2

Тақырып: реляциялық дерекқор және SQLModel ORM модельдері.

Мақсат

Бірінші жұмыстағы ER-сызбаны Python кластарына көшіру, кілттер мен объектілік байланыстарды баптау және SQLite базасына мысал жазбалар енгізу.

1. Модельдер

`app/database/models.py` файлында төрт кесте анықталады:

Модель	Кесте	Негізгі өрістер
Қонақ	`guests`	`id`, `full_name`, `phone`, `email`
Нөмір санаты	`room_categories`	`id`, `name`, `description`
Қонақүй нөмірі	`rooms`	`id`, `category_id`, `room_number`, `price_per_night`, `capacity`
Брондау	`bookings`	`id`, `guest_id`, `room_id`, `check_in`, `check_out`, `status`

`id` — бастапқы кілт. `category_id`, `guest_id`, `room_id` — сыртқы кілттер. Қонақтың телефоны мен email-і, санат атауы және нөмір белгісі қайталанбауы тиіс. Ақша `Decimal` негізіндегі сандық өріспен беріледі, күндер `date` түрінде сақталады.

`Relationship` арқылы бір санаттың нөмірлеріне, бір қонақтың брондауларына және бір нөмірдің брондауларына Python объектілері ретінде қатынауға болады. Бұл өрістердің өзі SQL кестесіне бөлек баған болып қосылмайды.

ИИ сұрауының үлгісі:

> *ER-сызбаға сәйкес Guest, RoomCategory, Room, Booking SQLModel модельдерін жаса. Бастапқы және сыртқы кілттерді, бірегей өрістерді, бірден көпке Relationship байланыстарын көрсет. Бағаға Decimal, брондау күндеріне date қолдан. Кесте атаулары guests, room_categories, rooms, bookings болсын.*

2. Дерекқорды қосу

`app/database/db.py` файлындағы `engine` дерекқорға қосылымдарды басқарады. Әдепкі URL:

```
sqlite:///./database.db
```

`init_db()` функциясы `SQLModel.metadata.create_all(engine)` көмегімен жоқ кестелерді жасайды. SQLite үшін сыртқы кілттерді тексеру қосылады. `DATABASE_URL` арқылы бөлек тестілік база таңдауға болады.

Кестелер схемасын өзгерткенде `create_all()` бұрыннан бар бағандарды автоматты көшірмейді. Бұл оқу жобасында бастапқы схеманы қайта жасау үшін бөлек жаңа база қолдануға болады; құнды деректері бар базада миграция қажет.

3. Бастапқы деректер

Жобаның түбірінде:

```
python seed.py
```

Скрипт алдымен кестелерді дайындайды, содан соң әр кестеге 5 тестілік жазба енгізеді. Жазбалар жалған мысал деректер ретінде қолданылады. Бірдей бастапқы деректермен қайта орындау жазбаларды көбейтпейді. Бұрынғы деректер өңделген не жойылған базаға қайта іске қосқанда `python seed.py --if-empty` қолданыңыз: ол деректері бар базаны толықтырмайды және өзгерістерді сақтайды.

SQL командаларын көру, Linux/macOS:

```
SQL_ECHO=1 python seed.py
```

Windows PowerShell:

```
$env:SQL_ECHO = "1"
python seed.py
Remove-Item Env:SQL_ECHO
```

Логта кестелерді тексеру/жасау және алғашқы іске қосуда деректерді енгізу сұраулары көрінеді. Кесте бұрын жасалған болса, әр іске қосуда `CREATE TABLE` қайталануы міндетті емес.

4. Git-те сақтау

`.gitignore` ішінде кемінде `venv/`, `.venv/`, `*.db`, `.env`, `__pycache__/` болуы керек. Код сақталады, ал база мен виртуалды орта қайта құрылады.

```
git add app/database/models.py app/database/db.py seed.py .gitignore
git commit -m "feat: add SQLModel ORM models and database seed script"
git push origin main
```

Қорғауда көрсету

`python seed.py` командасын орындау, сыртқы кілттерді және `Relationship` мағынасын түсіндіру. Бір кестеге қатысты Python класын ER-сызбадағы блокпен салыстыру. Бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №3

Тақырып: FastAPI арқылы серверлік REST API жасау.

Мақсат

Дерекқордағы мәліметті HTTP арқылы оқу және қосу, сұрауларды валидациялау, әр сұрауға бөлек сессия беру және Swagger UI арқылы API-ді көрсету.

1. Қолданбаны іске қосу

Виртуалды орта белсенді болуы керек. Алдымен мысал деректерді дайындаңыз:

```
python seed.py --if-empty
uvicorn app.main:app --reload
```

`app/main.py` FastAPI қолданбасын жасайды, lifespan кезінде `init_db()` шақырады және роутерлерді қосады. `GET /` сервердің күйін хабарлайды.

Дұрыс мекенжайлар:

- сервер: <http://127.0.0.1:8000/>;
- Swagger: <http://127.0.0.1:8000/docs>.

Бастапқы нұсқаулықтағы `http://127.0.0` толық емес; стандартты uvicorn порты — `8000`, Swagger жолы — `/docs`.

2. Сессияны тәуелділік арқылы алу

`app/database/db.py` ішіндегі `get_session()` функциясы сессияны ашады, `yield` арқылы маршрутқа береді және сұраудан кейін жабады. Маршрутта `Depends(get_session)` қолданылады. Барлық сұрауға бір ортақ ашық `Session` объектісін пайдалану дұрыс емес: транзакциялар мен сұраулар араласып кетуі мүмкін.

3. GET маршруттары

`app/routers/` бөлімінде:

- `GET /rooms` — нөмірлер тізімі;
- `GET /rooms/{room_id}` — ID бойынша нөмір;
- жоқ ID үшін `404 Not Found`.

Қосымша `/guests`, `/categories`, `/bookings` маршруттары байланысты деректерді дайындауға және көрсетуге арналған.

ИИ сұрауының үлгісі:

> *FastAPI және SQLModel көмегімен нөмірлер тізімін және ID бойынша бір нөмірді қайтаратын GET маршруттарын жаз. Сессияны Depends(get_session) арқылы ал. Жазба табылмаса HTTPException 404 қайтар. Жауап схемасын ORM моделінен бөлек анықта.*

4. POST және валидация

`app/schemas.py` ішінде клиенттен келетін дерекке арналған құру схемалары және жауап схемалары бөлек беріледі. Клиент бастапқы кілтті өз бетімен орнатпайды.

`POST /rooms` мысалы:

```
{
  "category_id": 1,
```

```
  "room_number": "DEMO-601",
  "price_per_night": "22000.00",
  "capacity": 2
}
```

`category_id=1` базада болуы керек. Сәтті қосылған жазба `201 Created` күйімен қайтады. Теріс баға, бос атау/белгі немесе жарамсыз сыйымдылық `422` қатесін беруі керек. Бірегей нөмір белгісінің қайталануы — `409 Conflict`.

ИИ сұрауының үлгісі:

> *Нөмір қосатын POST /rooms маршрутын жаса. Баға теріс болмасын, нөмір белгісі бос болмасын, сыйымдылық кемінде бір болсын. Санат бар екенін тексер. Жазбаны session.add/commit/refresh арқылы сақтап, 201 мәртебесін қайтар. Қате транзакцияда rollback жаса.*

5. Swagger арқылы көрсету

1. `/docs` ашып, `GET /rooms → Try it out → Execute` таңдаңыз.
2. `GET /rooms/{room_id}` ішінде тізімдегі ID-ді көрсетіңіз.
3. Теріс бағамен жаңа нөмір қосуға тырысыңыз: күтілетін код — `422`.
4. Жарамды дерек жіберіңіз: `201` және жасалған нөмірдің ID-і қайтуы керек.
5. Жоқ ID-ге GET жіберіңіз: күтілетін код — `404`.

Қолданбаның нақты түсірілген Swagger суреті: `docs/screenshots/swagger.png`. Тексерудің нақты қорытындылары `docs/verification.md` файлында сақталады.

Git-те сақтау

```
git add app/
git commit -m "feat: implement FastAPI endpoints for GET and POST with validation"
git push origin main
```

Бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №4

Тақырып: толық CRUD, ішінара өзгерту, жою, іздеу және сүзгілеу.

Мақсат

№3 жұмыстағы оқу және қосу әрекеттерін толықтырып, деректерді қауіпсіз өзгерту мен жоюды, query параметрлерімен іздеуді және қате жағдайларды өңдеуді жүзеге асыру.

1. PATCH схемасы

Жаңарту схемасындағы өрістерді өткізіп жіберуге болады. `model_dump(exclude_unset=True)` клиент жібермеген өрістерді жаңартуға қоспайды. Сондықтан тек бағаны жібергенде нөмір белгісі, санаты мен сыйымдылығы сақталады.

`Optional` немесе әдепкі `None` мәні деректер базасындағы міндетті өрісті автоматты қорғамайды. Клиент міндетті өріске нақты `null` жіберсе, схема оны валидация кезінде қабылдамауы тиіс. Бұл «өрісті жібермеу» мен «өріске null жіберу» арасындағы айырма.

2. Нөмірді өзгерту

`PATCH /rooms/{room_id}` денесі:

```
{
  "price_per_night": "24500.00"
}
```

Маршрут ID бойынша жазбаны іздейді. Жоқ болса `404` қайтарады. Бар болса жіберілген өрістерді ғана жаңартып, өзгерісті базаға сақтайды. `GET` арқылы бағаның өзгергенін және қалған өрістердің сақталғанын салыстыруға болады.

ИИ сұрауының үлгісі:

> *PATCH /rooms/{room_id} жаса. Бастапқы жазба табылмаса 404 бер. Тек жіберілген өрістерді model_dump(exclude_unset=True) және sqlmodel_update арқылы өзгерт. Теріс баға мен міндетті өрістерге null қабылдама. Санатты ауыстырғанда сыртқы кілттің бар екенін тексер.*

3. Жою

`DELETE /rooms/{room_id}`:

- жоқ ID — `404`;
- байланысқан брондауы бар нөмір — `409`;
- жоюға болатын нөмір — `204 No Content`.

`204` жауабының денесі болмайды. Нөмірдің брондаулары бар кезде оны бірден жоймау байланысқан деректердің жоғалуын болдырмайды.

Көрсету үшін алдымен брондаусыз жаңа нөмір қосып, соны жойыңыз. Seed деректеріндегі нөмірдің брондауы болуы мүмкін.

4. Іздеу және баға бойынша сүзгілеу

`GET /rooms` параметрлері:

Параметр	Қызметі
`search`	Нөмір белгісінің бір бөлігін іздеу
`min_price`	Ең төмен түндік баға
`max_price`	Ең жоғары түндік баға
`category_id`	Бір санаттағы нөмірлер

Мысал:

```
http://127.0.0.1:8000/rooms?search=10&min_price=10000&max_price=30000
```

Фильтрлер `select(...).where(...)` түріндегі параметрленген SQLModel сұраулары арқылы орындалады. Пайдаланушы енгізген мәтін SQL жолына тікелей жалғанбайды. Теріс баға шегі немесе төменгі шектің жоғарғы шектен үлкен болуы қате сұрау ретінде тексеріледі.

5. Брондау бизнес ережелері

Брондауды жасау кезінде қонақ пен нөмірдің бар болуы тексеріледі. `check_out` күні `check_in` күнінен кейін болуы керек. Бір нөмірдің күшін жоймаған брондаулары `[check_in, check_out)` аралықтарында қабаттаспайды: бір қонақ шыққан күні келесі қонақ келе алады. Қайшылыққа `409` қайтарылады. `cancelled` брондау нөмірді сол мерзімге босатады.

6. Қолмен тексеру сценарийі

1. `GET /rooms` нәтижесін және бір нөмірдің бастапқы өрістерін қараңыз.
2. Іздеу мен баға шектерін беріп, нәтижеге тек сәйкес жазбалардың кіретінін тексеріңіз.
3. Жаңа нөмір құрып, оның тек бағасын PATCH арқылы өзгертіңіз.
4. Сол нөмірді GET арқылы қайта алып, қалған өрістерді салыстырыңыз.
5. Нөмірді жойыңыз; сол ID-ге PATCH қайталағанда `404` күтіледі.
6. Бір нөмірге қабаттасатын екі брондау жасаңыз: екіншісі `409` беруі тиіс.

Автоматты тексеру:

```
python -m pytest -q
```

Нақты орындалу нәтижелері тексеру есебінде беріледі. №4 жұмыстың тақырыбында PUT аталғанымен, нұсқаулықтың орындалу қадамдары мен бағалау критерийлері PATCH-ті талап етеді.

Git-те сақтау

```
git add app/ tests/ docs/
git commit -m "feat: complete CRUD operations with PATCH, DELETE and query filtering"
git push origin main
```

Қорғауда PATCH пен PUT айырмасын, `exclude_unset=True` қызметін және жоқ ID-ге жауапты түсіндіру керек. Бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №5

Тақырып: Streamlit арқылы пайдаланушы веб-интерфейсін жасау.

Мақсат

№1–4 жұмыстарда дайындалған FastAPI серверіне HTTP сұраулар жіберетін интерфейс құру: мәліметті кестемен көрсету, динамикалық іздеу, жазба қосу және өшіру.

1. Интерфейс файлы және іске қосу

Негізгі файл — `app/frontend/ui.py`. Алдымен жоба түбірінде белсенді виртуалды ортамен деректерді дайындап, API-ді іске қосыңыз:

```
python seed.py --if-empty
uvicorn app.main:app --reload
```

Екінші терминалда сол ортаны белсендіріп:

```
streamlit run app/frontend/ui.py
```

Интерфейс: <http://localhost:8501>. Бірінші терминалды жаппаңыз. Бұрынғы `streamlit run frontend/app.py` командасы үйлесімділік файлы арқылы сол қолданбаны іске қосады.

ИИ сұрауының үлгісі:

> *StayBook қонақүй брондау жүйесі үшін Streamlit интерфейсін жаса. Бет тақырыбы мен sidebar қос, API күйін көрсет. Нөмірлер, брондаулар, қонақтар мен санаттарды бөлек бөлімдерге бөл. API-ге requests арқылы қатына.*

2. Requests арқылы деректерді алу

`requests.get(...)` API-ден JSON деректерін алады. Нөмірлер тізімі `/rooms`, санаттар `/categories`, қонақтар `/guests`, брондаулар `/bookings` арқылы сұралады. Тізім `st.dataframe` кестесінде көрсетіледі.

API адресі орта айнымалысынан алынады:

```
BASE_URL = os.getenv("BACKEND_URL", "http://127.0.0.1:8000")
```

Жергілікті ортада әдепкі мән жеткілікті, Docker ішінде `BACKEND_URL=http://backend:8000` беріледі. UI дерекқорға тікелей қосылмайды: барлық дерек әрекеттері API арқылы өтеді.

Сұрауларда күту уақытының шегі беріледі. Сервер қолжетімсіз болғанда интерфейс байланыс қатесін түсінікті хабарламамен көрсетеді. HTTP `404`, `409`, `422` сияқты жауаптардағы backend түсіндірмесі пайдаланушыға жеткізіледі.

3. Динамикалық іздеу және сүзгілер

Нөмір белгісін іздеу үшін `st.text_input` қолданылады. Іздеу виджеті дерек қосу формасынан тыс орналасады: пайдаланушы өзгерткен мән Streamlit-ке жіберілгенде скрипт қайта орындалып, жаңа `search` параметрімен GET сұрауы жасалады. Бөлек «іздеуді қолдану» формасын жіберу талап етілмейді.

Мысал сұрау:

```
requests.get(
    f"{BASE_URL}/rooms",
    params={"search": search_query, "min_price": min_price},
```

```
    timeout=10,
)
```

`params` URL-дегі query параметрлерін кодтайды. Нөмірдің толық мазмұнын жібермей, тізімге арналған шарттар ғана беріледі. Санат пен баға шектері де №4 жұмыстың API сүзгілерін қолданады.

4. Қосу, өзгерту және жою

Жаңа нөмір қосу формасы `st.form` арқылы беріледі. Пайдаланушы нөмір белгісін, санатын, бағасын және сыйымдылығын таңдайды. Сақтау батырмасы:

1. `POST /rooms` сұрауын `json=...` денесімен жібереді.
2. Сервердің валидация нәтижесін қарайды.
3. Сәтті болса `st.success` хабарламасын көрсетіп, мәліметті қайта жүктейді.

Жою таңдалған жазбаның ID-імен `DELETE` сұрауы арқылы орындалады. Қонақүй нөмірінің байланысқан брондауы болса, сервер оны жоюға жол бермейді. PATCH арқылы өзгерту де UI-ден қолжетімді.

ИИ сұрауының үлгісі:

> *Нөмір қосатын st.form жаса: room_number, category_id, price_per_night, capacity. Сақтағанда requests.post(..., json=payload) шақыр. Қате жауапты көрсет, ал сәтті қосылғанда тізімді жаңарт. Таңдалған нөмірді өшіру және тек өзгерген өрістерін PATCH арқылы сақтау мүмкіндігін қос.*

5. Қолмен тексеру

1. UI-де бастапқы нөмірлердің көрінуін тексеріңіз.
2. Нөмір белгісінің бір бөлігін енгізіп, сәйкес жолдарды қараңыз.
3. Баға шектерін және санатты өзгертіңіз.
4. Seed ішінде жоқ `DEMO-601` сияқты белгімен жаңа нөмір қосыңыз.
5. Оның бағасын өзгертіп, кейін брондаусыз жазбаны жойыңыз.
6. API-ді уақытша тоқтатып, UI-дегі байланыс қатесі хабарламасын тексеріңіз; кейін серверді қайта іске қосыңыз.

Нақты орындалған сценарийлер мен нәтижелер verification.md файлында беріледі.

Git-те сақтау

```
git add app/frontend/ frontend/ requirements.txt
git commit -m "feat: design interactive web UI using Streamlit integrated with FastAPI"
git push origin main
```

Бақылау сұрақтарының бес жауабы осында. Бастапқы құжатта осы жұмыстың соңғы екі сұрағы №6 жұмыстан кейін орналасқан; жауаптар тақырыбына сәйкес біріктірілді.

Зертханалық жұмыс №6

Тақырып: Docker және Docker Compose арқылы компоненттерді контейнерлеу.

Мақсат

FastAPI мен Streamlit-ті екі жеке контейнерге бөлу, ортақ ішкі желіде байланыстыру және бүкіл жүйені бір командамен іске қосу. SQLite деректерін контейнер өмірінен тәуелсіз сақтау.

1. Backend Dockerfile

Түбірдегі `Dockerfile` Python slim образына негізделеді. Алдымен тәуелділіктер файлы көшіріліп, пакеттер орнатылады, одан кейін код қосылады. Осындай реттілік бастапқы код қана өзгергенде пакеттердің орнатылу қабатын кэштен қайта пайдалануға мүмкіндік береді.

Қолданба `uvicorn app.main:app --host 0.0.0.0 --port 8000` арқылы контейнердің барлық интерфейсінде тыңдайды. `EXPOSE 8000` порт туралы метадерек береді; хостқа ашу Compose ішіндегі `ports` арқылы орындалады.

ИИ сұрауының үлгісі:

> *Python slim негізінде FastAPI үшін Dockerfile жаса. Алдымен тәуелділіктерді орнатып, кейін бастапқы кодты көшір. Серверді 0.0.0.0:8000 адресінде іске қос. SQLite файлы үшін /data каталогын қолдануға және сыртқы volume қосуға мүмкіндік бер.*

2. Frontend Dockerfile

`Dockerfile.frontend` Streamlit-ті іске қосады. Негізгі файл — `app/frontend/ui.py`. Тыңдау адресі `0.0.0.0`, порт `8501`. UI сервері API-ге `requests` арқылы қатынасады; SQLite файлын өзі ашпайды.

3. Compose сервисі және желі

`docker-compose.yml` екі сервисті біріктіреді:

Сервис	Қызметі	Хост порты
`backend`	FastAPI, SQLite, бастапқы деректер	`8000:8000`
`frontend`	Streamlit UI	`8501:8501`

Frontend үшін `BACKEND_URL=http://backend:8000` беріледі. `backend` — Compose желісіндегі сервис атауы, оны Docker-дің ішкі DNS жүйесі контейнер адресіне сәйкестендіреді. Контейнердегі `localhost` сол контейнердің өзін білдіреді; сондықтан frontend басқа контейнердегі API-ге `localhost:8000` арқылы қатынамайды.

`depends_on` пен backend healthcheck көмегімен frontend сервер дайын болғаннан кейін басталады. Бұл API іске қосылып үлгермеген кездегі алғашқы сұрау қатесін азайтады. Кейінгі желі не сервер қатесін UI бәрібір өңдеуі керек.

Compose файлы нұсқаулыққа сәйкес `version: '3.8'` өрісін сақтайды. Заманауи Compose v2 бұл өрістің ескіргені туралы хабарлама беруі мүмкін; ол құрастыру қатесі емес. Қазіргі Compose Specification үшін сервис, желі және volume параметрлері маңызды.

4. База деректерін сақтау

SQLite файлы `/data/database.db` жолында, named volume-ға байланыстырылған каталогта сақталады. Контейнерлік URL — `sqlite:////data/database.db`. Контейнерді ауыстыру немесе кәдімгі `docker compose down` орындалуы бұл volume-ды жоймайды. `docker compose down -v` volume-ды жояды, сондықтан оны деректерді сақтау қажет болғанда орындамаймыз.

Базаның контейнерлік жолы `DATABASE_URL` арқылы беріледі. Контейнер іске қосылғанда `python seed.py --if-empty` төрт кесте де бос болса ғана алғашқы 5 мысал жазбаны әр кестеге енгізеді. Кемінде бір кестеде дерек бар болса, қайта іске қосу бизнес деректерін қозғамайды. Бос базаға арналған қауіпсіз тексеруді қолмен де орындауға болады:

```
docker compose exec backend python seed.py --if-empty
```

5. Бір командамен іске қосу

Docker Desktop іске қосылғанын тексеріңіз. Бұрын жергілікті Uvicorn/Streamlit жұмыс істеп тұрса, оларды `Ctrl+C` арқылы тоқтатыңыз.

```
docker compose up --build
```

Ескі Compose v1 орнатылған ортада баламасы — `docker-compose up --build`. Жаңа ортада `docker compose` қолданылады.

- UI: <http://localhost:8501>
- Swagger: <http://localhost:8000/docs>

Күйді және журналдарды тексеру:

```
docker compose ps
docker compose logs --tail=80 backend frontend
```

Тоқтату және контейнерлерді қайта құру:

```
docker compose down
docker compose up --build
```

UI арқылы жазба қосып, қалыпты `down` және қайта іске қосудан кейін оның сақталғанын тексеріңіз. Нақты Docker тексеруінің жағдайы verification.md файлында тіркеледі.

6. Құрастыруға кірмейтін файлдар

`.dockerignore` виртуалды ортаны, `.git`, жергілікті `*.db`, кэштер мен қажетсіз артефактілерді образға қоспайды. Бұл контекст көлемін азайтып, жергілікті құпия конфигурацияның образға кездейсоқ енуін болдырмайды.

```
git add Dockerfile Dockerfile.frontend docker-compose.yml .dockerignore app/
git commit -m "chore: containerize application with Docker and Docker Compose"
git push origin main
```

Бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №7

Тақырып: автоматты тесттер, кодты тексеру және жобаны кешенді қорғау.

Мақсат

API мен бизнес логиканың жұмысын автоматты тексеру, қате жағдайларды бақылау, соңғы README мен есепті дайындау және жүйені толық іске қосып көрсету.

1. Тест ортасы

Виртуалды ортада:

```
python -m pip install -r requirements-dev.txt
```

`tests/` ішінде `test_api.py` және қосымша сценарийлер бар. Pytest тест функцияларын табады; FastAPI `TestClient` сұрауларды қолданбаға жібереді; HTTPX клиенттік HTTP әрекеттеріне негіз болады. TestClient үшін Uvicorn-ды жеке іске қосу қажет емес.

Тест сессиялары `Depends(get_session)` тәуелділігін ауыстыру арқылы жеке тестілік дерекқорды пайдаланады. Қолданбаның негізгі жұмыс базасын тест барысында тазалауға болмайды.

ИИ сұрауының үлгісі:

> *FastAPI қолданбасы үшін TestClient қолданатын pytest интеграциялық тесттерін жаса. GET /rooms 200 және тізім қайтарсын. Жарамды POST /rooms 201 және жасалған жазба қайтарсын. Бөлек тестілік база мен dependency_overrides қолдан. Теріс баға, жоқ ID, PATCH-тегі null, брондау қабаттасуы және сыртқы кілт қателері үшін сценарий қос.*

2. Тесттерді іске қосу

```
pytest
```

Қысқа шығару үшін:

```
python -m pytest -q
```

Негізгі тексерілетін әрекеттер:

- GET тізімі және ID бойынша оқу;
- POST және `201` жауабы;
- PATCH бір өрісті өзгертіп, басқаларын сақтауы;
- DELETE және жоқ ID үшін `404`;
- баға, бос мәтін, міндетті өрістер және `null` валидациясы;
- сүзгілердің жұмысы мен баға шектерінің дұрыстығы;
- бірегей мәндердің қайталануына `409`;
- байланысқан негізгі жазбаны жоюға шектеу;
- брондау күндері мен қабаттасуын тексеру;
- seed скриптінің қайталап орындауға жарамдылығы.

Нақты тест саны мен өткен/өтпеген нәтижелер verification.md файлында берілген. Құжаттағы сценарийлер тізімі тест орындалды дегеннің орнына жүрмейді: қорғауда команданы орындап, нақты нәтижені көрсету керек.

3. Код ревьюі

ИИ сұрауының үлгісі:

> *Backend кодын қарап шық. Сессиялардың жабылуын, транзакция rollback-ын, SQL сұрауларының параметрленуін, PATCH кезінде null/жіберілмеген өріс айырмасын, сыртқы кілттерді және брондау қабаттасуының жарысу жағдайын тексер. Нақты кодқа сүйенген түзетулер мен оларды дәлелдейтін тесттер ұсын.*

Жобада осы мәселелерге арналған тәсілдер:

Мәселе	Кодтағы тәсіл
Сессияның ашық қалуы	`with Session(...)` және `yield`
Қате жазудан кейінгі транзакция	`IntegrityError` өңдеу, `rollback()`
SQL мәтініне пайдаланушы енгізуін жалғау	SQLModel `select`/`where`, параметрленген шарттар
PATCH міндетті өрісті босатуы	Жіберілмеген өрістерді алып тастау және нақты `null`-ды қабылдамау
Жоқ сыртқы кілт	Қонақ, нөмір және санаттың бар болуын тексеру
Брондаулардың қабаттасуы	Мерзім шарттары және транзакциямен қорғалған жазу

ИИ қорытындысы әзірлеушінің тексеруін алмастырмайды. Өзгерісті қабылдау үшін код логикасы мен орындалған тест нәтижесі бағаланады.

4. Репозиторий мен есеп

README бастапқы нұсқаулықтағы бес негізгі бөлімге сәйкес: пәндік сала, ER-архитектура, технологиялар, іске қосу және тесттер. Қосымша жеті зертхана картасы, код құрылымы және қорғау сценарийі берілген.

Тапсырылатын материалдар:

- GitHub репозиторийі (https://github.com/kasanovaaruzan74-oss/StayBook);
- ER-диаграмма, SVG және draw.io файлдары;
- толық Markdown есебі;
- Word есебі;
- PDF есебі;
- 35 бақылау сұрағының жауабы;
- тексеру нәтижелері.

Автор — Касанова Аружан, тобы — Ис-24а. Оқытушы мен тапсыру күні енгізілмегендіктен, есепте тиісті орындар толтыруға қалдырылған. Коммиттер зертхана кезеңдерімен бөлінеді және олардың нақты жасалған уақыты сақталады.

```
git add .
git commit -m "test: add integration tests and final documentation update"
git push origin main
```

5. Соңғы қорғау сценарийі

1. Docker Desktop іске қосылғанын тексеріңіз. Жергілікті серверлерді тоқтатып, порттарды босатыңыз.
2. `docker compose down` және `docker compose up --build` арқылы контейнерлерді қайта іске қосыңыз. Дерек volume-ын жоймаңыз.

3. `http://localhost:8501` ашып, бастапқы кестені және іздеу нәтижесін көрсетіңіз.
4. Жаңа нөмір қосып, оның бағасын өзгертіңіз; кейін брондаусыз нөмірді жойыңыз.
5. Қонақ пен нөмірді таңдап, дұрыс брондау жасаңыз. Сол нөмірге қабаттасатын мерзіммен брондаудың қабылданбайтынын көрсетіңіз.
6. `http://localhost:8000/docs` арқылы Swagger маршруттарын және `201/404/409/422` жауаптарын түсіндіріңіз.
7. Белсенді виртуалды ортада `pytest` орындаңыз.
8. ER-сызбадағы байланыстарды, модульдердің міндетін және Git тарихын түсіндіріңіз.

Осы жұмыстың бес бақылау сұрағы бастапқы құжатта №7 тақырыбына дейін орналасқан. Жауаптары осында.

Қорғауға арналған бақылау сұрақтары

Жауаптар бастапқы нұсқаулықтағы №1–7 жұмыстардың 35 бақылау сұрағына сәйкес реттелді. Мысалдар StayBook жобасына бейімделді.

Зертханалық жұмыс №1

1. Python жобасында виртуалды орта не үшін керек? Оны құрмасақ не болады?

`venv` бір жобаның Python пакеттерін басқа жобалардың пакеттерінен бөледі. StayBook тәуелділіктері сол ортаға орнатылады. Онсыз жүйелік Python ортасында кітапхана нұсқалары қайшы келуі, бір жобаны жаңарту екіншісін бұзуы мүмкін. Виртуалды орта Python нұсқасын өзі орнатпайды, ол таңдалған Python интерпретаторының негізінде жасалады.

2. `.gitignore` не үшін қажет? Неліктен оған `venv` және парольдері бар файлдар қосылады?

`.gitignore` Git-ке әлі тіркелмеген қандай файлдарды бақылауға алмау керегін көрсетеді. Виртуалды орта үлкен әрі басқа компьютерде қайта орнатылады, ал құпия конфигурация жарияланбауы тиіс. Жобаға `venv/`, `.venv/`, `.env`, `*.db`, `__pycache__/` кірмейді. Бұрын Git-ке қосылған файлды `.gitignore`-ға жазу оны тарихтан автоматты алып тастамайды.

3. Cursor немесе ИИ плагині қандай қызмет атқарады? ИИ кодын тексермей көшірудің тәуекелі қандай?

ИИ көмекшісі жоба құрылымын ұсынуға, модель мен API кодын жазуға, қатені түсіндіруге және тест дайындауға көмектеседі. Әзірлеуші берілген кодтың логикасын тексереді, қолданылған кітапхананың құжатымен салыстырады және іске қосады. Тексерілмеген код ескірген API-ді, қате байланыстарды, қауіпсіз емес сұрауларды немесе орындалмайтын функцияларды қамтуы мүмкін.

4. Git тармағы деген не? Неліктен командалар негізгі тармаққа тікелей жазуды шектейді?

Тармақ — коммиттер тарихындағы бөлек бағытты көрсететін сілтеме. `git branch feature/room-filters` жаңа тармақ жасайды; `git switch -c feature/room-filters` оны жасап, соған ауыстырады. Командаларда `main` жиі қорғалады: өзгеріс бөлек тармақта жазылып, pull request пен тексерулерден кейін біріктіріледі. Бұл барлық жобадағы Git-тің міндетті тыйымы емес, репозиторийдің жұмыс тәртібі.

5. `git commit` пен `git push` айырмасы неде? Өзгерістер қайда сақталады?

`git commit` индекске қосылған өзгерістердің нұсқасын жергілікті `.git` тарихына сақтайды. `git push` жергілікті коммиттерді GitHub сияқты қашықтағы репозиторийге жібереді. Тек commit жасалса, өзгеріс басқа компьютерлерде әлі көрінбейді. `git add` файлдарды келесі коммитке дайындайды.

Зертханалық жұмыс №2

1. ORM деген не және ол қандай мәселені шешеді?

ORM — Object-Relational Mapping, Python объектілерін реляциялық кестелермен сәйкестендіру. SQLModel ішінде `Room` класының объектісі `rooms` кестесінің жазбасына сәйкес келеді. Әзірлеуші негізгі дерек әрекеттерін Python кластары және сұрау құрылымдары арқылы орындайды. Дегенмен SQL, транзакция және дерекқор байланыстарын түсіну қажет болып қалады.

2. `category_id` пен `category` Relationship өрісінің айырмасы қандай?

`category_id` — `rooms` кестесінде сақталатын нақты сыртқы кілт бағаны. Ол `room_categories.id` мәніне сілтейді. `category` — тиісті санат объектісіне қатынауға арналған ORM байланысы; оның атымен қосымша SQL бағаны жасалмайды. Бірінші өріс кілт мәнін, екіншісі Python объектілері арасындағы қатынасты береді.

3. Seed деректер не үшін қажет? Бос базамен жұмыс неге ыңғайсыз?

Seed — жүйені алғашқы үлгі деректермен толтыру. Қонақтар, санаттар, нөмірлер мен брондаулар болғанда API тізімін, сыртқы кілттерді, сүзгілерді және қайшылықтарды бірден тексеруге болады. Бос база дұрыс сүзгі мен қате сүзгінің екеуіне де бос тізім қайтарып, мәселені жасыруы мүмкін. StayBook-та `python seed.py` бастапқы деректерді енгізеді, қайта іске қосу бірдей жазбаларды көбейтпейді.

4. `create_engine` ішіндегі `echo=True` қандай рөл атқарады?

Бұл параметр орындалатын SQL сұрауларын және транзакция логтарын консольге шығарады. Кесте жасалғанын, дұрыс `SELECT` немесе `INSERT` орындалғанын көруге көмектеседі. StayBook-та оны `SQL_ECHO=1` арқылы қосуға болады. Логты қарағанда енгізілген деректердің де көрсетілуі мүмкін екенін ескереміз.

5. Неліктен `database.db` Git-ке кірмейді, ал `models.py` міндетті түрде кіреді?

`models.py` деректер құрылымын анықтайтын бастапқы код, сондықтан өзгерістері бақыланады. `database.db` — жұмыс барысында өзгеретін жергілікті деректер файлы; ол көлемді болуы, жеке мәлімет қамтуы және біріктіру қайшылығын туғызуы мүмкін. Құрылым `init_db()`, ал мысал мазмұн `seed.py` арқылы қайта құрылады.

Зертханалық жұмыс №3

1. REST API деген не? GET пен POST айырмасы қандай?

REST API ресурстарға HTTP арқылы қатынауды ұйымдастырады. StayBook-та ресурстар — `/rooms`, `/guests`, `/bookings`. `GET` деректі оқиды және сервер күйін өзгертпеуі тиіс. `POST` көбіне жаңа ресурс жасайды; сұрау денесіне жаңа жазбаның деректері жіберіледі және сәтті жасағанда `201 Created` қайтады.

2. FastAPI ішіндегі Pydantic не істейді? Ол қате енгізуден қалай қорғайды?

Pydantic сұрау өрістерінің типтерін, шектерін және өзара шарттарын тексереді, сондай-ақ жауап құрылымын сипаттауға көмектеседі. Мысалы, теріс `price_per_night`, нөлдік `capacity` немесе бос `room_number` қабылданбайды. Схемаға сәйкес емес

сұрауға FastAPI `422` қайтарады. Бұл логин мен пайдаланушы құқықтарын тексеруді алмастырмайды.

3. `Depends` деген не? Неге барлық маршрутқа бір ортақ сессияны импорттамаймыз?

`Depends` — тәуелділікті енгізу механизмі. `Depends(get_session)` маршрутқа оның сұрауына арналған сессияны береді. Генератор сессияны ашып, өңдеуден кейін жабады. Бір ортақ ұзақ өмір сүретін `Session` параллель сұраулардың транзакцияларын араластырып, қате күй қалдыруы мүмкін. Ал `engine` объектісін ортақ пайдалану қалыпты: engine мен Session бір ұғым емес.

4. Swagger UI қалай жасалады? Оның HTML кодын қолмен жазу қажет пе?

FastAPI маршруттардан, типтерден және Pydantic схемаларынан OpenAPI спецификациясын жасайды. Дайын Swagger UI сол спецификацияны қолданып, интерактивті құжаттаманы көрсетеді. Қолмен HTML жазу қажет емес. StayBook үшін мекенжай — `http://127.0.0.1:8000/docs`, спецификация — `/openapi.json`.

5. `422 Unprocessable Entity` қашан қайтады?

Сұрау форматы түсінікті болғанымен, оның өрістері қабылдау схемасына сәйкес болмағанда `422` қайтады. Мысалы, теріс баға, жарамсыз күн, міндетті өрістің болмауы немесе міндетті өріске `null` жіберу. Бұл `404`-тен өзгеше: `404` дұрыс пішіндегі ID-мен ізделген ресурс табылмағанын білдіреді.

Зертханалық жұмыс №4

1. PUT пен PATCH айырмасы қандай? Қайсысы трафикті азайтуы мүмкін?

`PUT` ресурстың толық көрінісін ауыстыруға, `PATCH` оның бір бөлігін өзгертуге арналған. Нөмірдің тек бағасын өзгерту үшін PATCH сұрауымен бір өріс жіберу жеткілікті, сондықтан мұндай жағдайда трафик аз болады. Барлық жағдайда бір әдіс аз трафик береді деген ереже жоқ: ол жіберілетін дерек көлеміне байланысты.

2. Query параметрлері деген не және URL ішінде қалай беріледі?

Query параметрлері URL жолындағы `?` таңбасынан кейін жазылады, бірнешеуі `&` арқылы бөлінеді. Мысалы: `/rooms?search=10&min_price=10000&max_price=30000`. Олар ресурсқа қосымша іздеу не сүзгі шарттарын береді. Нөмірдің ID-і `/rooms/5` түрінде берілсе, ол query емес, path параметрі болады.

3. PATCH кезінде None мәндерінен қалай қорғанамыз?

`model_dump(exclude_unset=True)` клиент жібермеген өрістерді жаңарту сөздігінен алып тастайды. Бірақ клиент нақты `null` жіберген өріс бұл сөздікке кіреді. Сондықтан міндетті өрістерге `null` мәнін схема валидаторымен қабылдамаймыз; дерекқордың `NOT NULL` шектеуі қосымша қорғаныс береді. `Optional` жалғыз өзі бұл мәселені шешпейді. `exclude_none=True` қолдану барлық жағдайда балама емес: кейбір рұқсат етілген өрістерді клиент әдейі босатуы мүмкін.

4. Жою алдында ID бойынша жазбаны не үшін іздейміз?

Алдымен жазбаның бар-жоғын біліп, табылмаса түсінікті `404` қайтарамыз. Сонымен бірге бизнес шарттарын тексереміз: мысалы, брондауы бар нөмірді жоймау.

Базаның сыртқы кілттері мен транзакциялар да қажет; алдын ала іздеу дерекқор шектеулерін алмастырмайды.

5. CRUD деген не? Қандай HTTP әдістері сәйкес келеді?

Әрекет	Мағынасы	HTTP әдісі
Create	Жаңа жазба қосу	`POST`
Read	Тізімді немесе бір жазбаны оқу	`GET`
Update	Жазбаны толық немесе ішінара өзгерту	`PUT` немесе `PATCH`
Delete	Жазбаны жою	`DELETE`

StayBook-та ішінара жаңарту `PATCH` арқылы іске асырылады.

Зертханалық жұмыс №5

1. Streamlit пен FastAPI қалай байланысады? Қандай протокол және формат қолданылады?

Браузер Streamlit бетімен жұмыс істейді; Streamlit-тің Python процесі FastAPI серверіне HTTP сұраулар жібереді. Мәлімет көбіне JSON форматында беріледі. Мысалы, UI-дегі форма `requests.post(..., json=payload)` арқылы `/rooms` маршрутына жіберіледі, FastAPI деректі тексеріп, SQLModel сессиясы арқылы базаға сақтайды және JSON жауап қайтарады. Streamlit SQLite файлын тікелей ашпайды.

2. Requests не істейді? `params` пен `json` айырмасы қандай?

Requests — Python-нан HTTP сұраулар жіберетін кітапхана. `params={"search": "10"}` мәндері URL query жолына кодталады: `/rooms?search=10`. `json={"room_number": "DEMO-601", ...}` сұрау денесін JSON ретінде жібереді және тиісті Content-Type орнатады. Іздеу шарттары үшін `params`, жаңа объектінің өрістері үшін `json` қолданылады.

3. Backend өшсе, Streamlit интерфейсіне не болады?

Streamlit процесі жұмыс істеп тұруы мүмкін, бірақ API-ден жаңа дерек ала алмайды және жазба сақтай алмайды. Requests қосылым қатесін не timeout береді; өңделмесе traceback көрсетілуі мүмкін. Жобада мұндай қатені ұстап, сервермен байланыс жоқ екені туралы түсінікті UI хабарламасын беру қарастырылған. Backend қайтқан соң сұрауды қайталап, деректі жаңартуға болады.

4. Streamlit пайдаланушы әрекетін қалай өңдейді? Неге скрипт қайта орындалады?

Streamlit виджет мәні серверге өзгеріп жеткенде немесе батырма басылғанда, әдетте скриптті жоғарыдан төмен қайта орындайды. Осылайша интерфейс жаңа widget мәндеріне сәйкес қайта құрылады. Күйді сақтау үшін `st.session_state`, қажет есепті кэштеу үшін тиісті cache механизмдері қолданылады. `st.form` ішіндегі бірнеше өзгеріс әдетте жіберу батырмасына дейін біріктіріледі. Мәтін өрісінің мәні көбіне Enter басқанда не фокус ауысқанда жіберіледі; қайта орындалу әр перне басылуына міндетті түрде тең емес.

5. Streamlit-тің HTML/CSS/JavaScript тәсілімен салыстырғандағы артықшылықтары мен шектеулері қандай?

Streamlit Python-мен тез прототип, әкімшілік интерфейс және деректер панелін жасауға ыңғайлы: дайын виджеттер мен кестелер бар, бөлек JavaScript қажет емес. Шектеулері — бет құрылымы мен мінезін толық басқару мүмкіндігінің аздығы, rerun моделіне бейімделу және күрделі клиенттік логикаға қосымша компоненттердің қажет болуы. Күрделі өнімде дәстүрлі frontend көбірек бақылау береді, ал Streamlit осы оқу жүйесін тез көрсетуге қолайлы.

Зертханалық жұмыс №6

1. Контейнерлеу деген не? Docker контейнері виртуалды машинадан қалай ерекшеленеді?

Контейнер қолданбаны тәуелділіктерімен бірге оқшауланған процесс ортасында іске қосады. Бір Docker хостындағы Linux контейнерлері хосттың Linux ядросын ортақ пайдаланады; әр контейнерге жеке толық қонақ операциялық жүйе жүктелмейді. Виртуалды машина гипервизор үстінде бөлек қонақ жүйені және оның ядросын іске қосады. Windows/macOS-та Docker Desktop Linux контейнерлерін Linux виртуалды машинасының ішінде іске қосуы мүмкін — бұл контейнердің өзіне жеке ядро беріледі деген сөз емес.

2. Dockerfile ішіндегі EXPOSE, WORKDIR, COPY, CMD не істейді? RUN мен CMD айырмасы қандай?

`WORKDIR` кейінгі командалардың жұмыс каталогын белгілейді. `COPY` құрастыру контекстіндегі файлдарды образға көшіреді. `EXPOSE` қолданба тыңдайтын портты сипаттайды, бірақ портты хостқа автоматты ашпайды. `RUN` образ құрастырылған кезде орындалады, мысалы пакеттерді орнату. `CMD` контейнер іске қосылғанда әдепкі орындалатын команданы көрсетеді, мысалы Uvicorn не Streamlit.

3. `.dockerignore` не үшін керек? Жергілікті venv-ті образға көшірудің салдары қандай?

`.dockerignore` құрастыру контекстіне қажетсіз файлдарды қоспайды. Жергілікті виртуалды орта басқа операциялық жүйеге, архитектураға немесе Python жолына байланысқан болуы мүмкін. Оны көшіру образды үлкейтіп, жүйеге сәйкес келмейтін бинарлық пакеттерді әкелуі мүмкін. Контейнер өз тәуелділіктерін `requirements.txt` арқылы орнатады; кэш, база, `.git` және құпия жергілікті файлдар да сүзгіленеді.

4. Compose сервис атын нақты IP адреске қалай сәйкестендіреді?

Compose ортақ Docker желісін жасайды. Docker-дің сол желідегі ішкі DNS резолвері `backend` сияқты сервис атын контейнердің адресіне сәйкестендіреді. Сондықтан frontend `http://backend:8000` адресін қолданады; контейнердің IP-і өзгерсе де сервис аты сақталады. Браузердің хост жүйесінде `backend` адресі міндетті түрде шешілмейді: пайдаланушы жарияланған `localhost` портын ашады.

5. `docker compose down` SQLite деректеріне қалай әсер етеді? Дерек жоғалмауы үшін не керек?

Дерек тек контейнердің writable layer ішінде болса, контейнер жойылғанда файл да жоғалады. Named volume не bind mount қолданылса, дерек контейнерден тәуелсіз сақталады. StayBook-та SQLite файлы volume-ға қосылған каталогта орналасады.

Қалыпты `docker compose down` volume-ды сақтайды, ал `down -v` named volume-ды жояды. Volume қолдану сақтық көшірме жасауды алмастырмайды.

Зертханалық жұмыс №7

1. Автоматты Unit/Integration тесттер не үшін керек? Неліктен қолмен тестілеу жеткіліксіз бола бастайды?

Unit тест жеке функцияны не компонентті, интеграциялық тест бірнеше бөліктің бірлескен жұмысын тексереді. Жүйе өскен сайын барлық GET, POST, қате шарттар мен байланыстарды Swagger арқылы әр өзгерістен кейін қолмен қайталау ұзақ және қателесуге бейім болады. Автоматты тесттер бір сценарийлерді тұрақты қайталайды және өзгерістің мінез-құлықты бұзғанын жылдам көрсетеді. Қолмен UI тексеруі де қажет, бірақ ол автоматты тексеруді толықтырады.

2. FastAPI TestClient деген не? Тест үшін Uvicorn қажет пе?

`TestClient` HTTP тәрізді сұрауларды FastAPI ASGI қолданбасына тест процесінің ішінде жіберуге мүмкіндік береді. Сыртқы желілік порт ашып, Uvicorn-ды жеке іске қосу қажет емес. `with TestClient(app) as client:` lifespan кезеңдерінің орындалуын қамтамасыз етеді. Дерекқор тәуелділігін тестілік сессияға ауыстырсақ, нақты жұмыс деректеріне тимей маршруттарды тексереміз.

3. HTTPX асинхронды Python қолданбаларын тексеруде қандай рөл атқарады?

HTTPX — синхронды және асинхронды клиенттері бар HTTP кітапханасы. FastAPI/Starlette TestClient оның мүмкіндіктеріне сүйенеді. Асинхронды сценарийлерде `httpx.AsyncClient` және ASGI transport арқылы қолданбаны тест процесінің ішінде сұратуға болады. Requests интерфейстің API-ге қатынауына, ал HTTPX бұл жобаның тест ортасына қолданылады.

4. ИИ көмекшісі код ревьюі мен осал жерлерді іздеуде не береді?

ИИ бірнеше файлдың байланысын тез қарап, ұмытылған қате өңдеуді, қауіпті SQL құрастыруды, транзакция мәселесін немесе жетіспейтін тест жағдайын ұсына алады. Бұл ревьюге дайындықты және тест сценарийлерін ойластыруды жылдамдатады. Оның қорытындылары қате болуы не мәселені өткізіп алуы мүмкін, сондықтан әр ұсыныс кодпен, кітапхана құжатымен және нақты тестпен тексеріледі. ИИ барлық осалдық табылды деген кепілдік бермейді.

5. Регрессиялық тестілеу деген не? Осы тесттер кейінгі өзгерістерден қалай қорғайды?

Регрессиялық тестілеу бұрын дұрыс жұмыс істеген мінез-құлықтың жаңа өзгерістен кейін сақталғанын тексереді. Мысалы, жаңа брондау мүмкіндігін қосқанда бағаны PATCH арқылы өзгерту, бос өрісті қабылдамау немесе санат бойынша сүзгілеу бұзылуы мүмкін. Бұрын жазылған тесттер әр өзгерістен кейін қайта орындалып, осындай бұзылысты көрсетеді. Ақау табылса, оның сценарийін жеке тестпен бекіту сол қатенің қайта оралуын азайтады.

Нақты тексеру нәтижелері

Тексерілген күн: **2026-11-11 (UTC)**. Тексеру осы жобаның орындалатын кодында жүргізілді.

Автоматты тексеру

```
python -m pytest tests -q
38 passed in 4.34s
```

API интеграциялық тесттері әр тестке бөлек уақытша SQLite базасын қолданды. Қосымша Streamlit AppTest сценарийлері динамикалық іздеу, POST формасы және сервер қолжетімсіздігін тексерді.

Бағыт	Нәтиже
Төрт ORM кестесі, сыртқы кілттер және Relationship	Өтті
Seed-ті жеке командамен екі рет орындау	Әр кестеде 5 жазба; қайталанған жазба жоқ
Контейнердің seed --if-empty қайта іске қосылуы	Өңделген email мен жойылған брондау өзгеріссіз сақталады
Негізгі GET/POST TestClient тесттері	tests/test_api.py: 200 және 201
Streamlit автоматты іздеу және POST формасы	Қосымша іздеу батырмасынсыз query параметрлері жіберіледі; форма тізімді жаңартады
GET тізім және ID бойынша іздеу	Өтті; жоқ ID — 404
POST және деректерді валидациялау	Өтті; сәтті — 201, жарамсыз — 422
Бірегей нөмір және сыртқы кілтті тексеру	Өтті; қайталану — 409, жоқ байланыс — 404
PATCH кезінде тек берілген өрістерді өзгерту	Өтті; қалған өрістер сақталады, айқын null — 422
DELETE және жойылған ID-ге қайта сұрау	Өтті; жауап денесі жоқ 204, кейін 404
Іздеу, баға аралығы, санат және қате шектер	Өтті
Брондау мерзімдері және қабаттасу	Өтті; қайшылық — 409, қатарлас күндерге рұқсат
Брондауды болдырмау және қайта белсендіру	Өтті
Байланысқан нөмірді жоюдан қорғау	Өтті; 409

`pip check`: **No broken requirements found.** Барлық Python файлдарының синтаксисі тексерілді. Тәуелділіктің Starlette пакеті тест кезінде httpx интеграциясы туралы ескерту шығарды; тексерулер сәтті аяқталды.

Браузерде тексеру

Docker контейнерлерінде жұмыс істеп тұрған FastAPI және Streamlit серверлері Chromium браузерінде ашылды. Скриншоттар нақты жұмыс істеп тұрған беттерден алынды:

- Swagger жалпы көрінісі.
- GET /rooms: 200 жауабы.
- Терiс баға: POST /rooms → 422.
- Нөмірлер интерфейсі.
- Автоматты іздеу нәтижесі.
- Брондаулар интерфейсі.

Streamlit-тің төрт бөлімі AppTest арқылы да ашылып тексерілді. Браузерде нөмірлер мен брондаулар беттері қосымша тексерілді. Интерфейстен жаңа нөмір қосу және растау белгісімен өшіру толық орындалды; кейін сол ID-ге API сұрауы 404 қайтарды. Тексеру үшін қосылған уақытша нөмір тазартылды.

Docker тексеруі

Екі образ Python 3.11 негізінде нақты жиналды. Екі сервис те healthy, ішкі http://backend:8000 байланысы, CRUD және тұрақты volume тексерілді. Толық down/up циклінде енгізілген жазба мен өзгертілген email сақталды. Нақты нәтижелер Docker есебінде берілген.

Орындалу ортасы

Python 3.12.14, FastAPI 0.162.4, SQLModel 0.0.48, Streamlit 1.67.0, pytest 9.2.0. Орнатылған пакеттердің нақты нұсқалары `requirements.lock.txt` ішінде сақталған.

Бұл нәтижелер әзірлеу ортасындағы тексеруге жатады. Студенттің жеке компьютерінде Cursor/VS Code орнатылғаны немесе қорғауды өзі өткізгені туралы мәлімдеме жасалмайды. Жеке компьютерде іске қосу қадамдары README ішінде берілген.

Docker контейнерлерін нақты тексеру

Тексерілген күн: **2026-11-11 (UTC)**. Docker 29.9.2, Compose v2.40.3.

Тексеру	Нақты нәтиже
`docker compose config`	Конфигурация дұрыс
Backend образы	`python:3.11-slim` негізінде жиналды
Frontend образы	`python:3.11-slim` негізінде жиналды
Екі сервисті бірге іске қосу	Backend және frontend — healthy
`GET http://localhost:8000/`	HTTP 200
`GET http://localhost:8501/_stcore/health`	HTTP 200
Frontend → `http://backend:8000/rooms`	Docker ішкі DNS арқылы HTTP 200, 5 нөмір
Бастапқы база	Төрт кестенің әрқайсысында 5 жазба
Контейнердегі пайдаланушы	UID 10001, `/data` жазуға қолжетімді
Тірі API арқылы POST/PATCH/іздеу	201, 200 және сәйкес сүзілген нәтиже
Толық `docker compose down` → қайта іске қосу	Жаңа жазба және өзгертілген қонақ email-і сақталды
Бастапқы деректерді қайта қоспау	`seed.py --if-empty` бұрынғы деректерге тимеді

Образдар алдымен жеке Docker build командаларымен жиналды. Содан кейін Compose-тің мына командалары орындалды:

```
docker compose up --no-build --detach --wait
docker compose ps
docker compose down
docker compose up --no-build --detach --wait
```

Қайта іске қосу тексеруінде `-v` қолданылмады. Сондықтан `staybook_data` named volume-дағы SQLite файлы сақталды. Қалыпты компьютерде құрастыру мен іске қосуды біріктіретін команда: `docker compose up --build`.

Cloud-та образ құрастыру кезінде ортаның CA сертификаты уақытша BuildKit secret арқылы берілді; TLS тексеруі қосулы қалды. Dockerfile ішінде бұл secret міндетті емес: қалыпты жергілікті ортада қосымша сертификат қажет болмайды.

Тексеру аяқталған соң уақытша жазбалар жойылып, өзгертілген мысал email қалпына келтірілді. Демонстрация базасында әр кестеде қайтадан 5 жазба қалды.